LogPIM: Low-Overhead Logarithmic Arithmetic for HBM-Based Processing-in-Memory

Taewoon Kang, Jonghyun Lee, Yujin Kim, Murali Annavaram, and Gunjae Koo

***Abstract*—Processing-in-memory (PIM) reduces data movement cost, but integrating processing units into DRAM memory cells incurs substantial area overhead and reduces available memory capacity. We propose LogPIM, an LNS-based PIM architecture that replaces FP16 multipliers with dual-mode SIMD adders, performing LNS multiplication using fixed-point addition and FP16 accumulation. The proposed LUT controller performs LNS-to-FP16 conversion using one DRAM row per bank, while the ping-pong mechanism overlaps matrix and LUT accesses across paired banks to avoid repeated row switching. Compared to conventional HBM-PIM, LogPIM reduces processing-unit area and GeMV kernel energy by 30.1% and 33.9%, respectively. LogPIM achieves a $1.68\times$ speedup on average for PIM kernels with pre-encoded LNS16 inputs. The LNS16-based GeMV outputs by LogPIM achieve an average cosine similarity of 99.99% to the FP32 reference. Our evaluation demonstrates that LogPIM improves area, energy, and performance for HBM-based PIM.**

***Index Terms*—function-in-memory, processing-in-memory, near data processing, logarithmic number system**

I. Introduction

Processing-in-memory (PIM) reduces data movement and improves bandwidth utilization by offloading computation into memory. However, high-bandwidth memory (HBM)-based PIM replaces DRAM sub-arrays with processing units, reducing the capacity available for applications and models (Fig. 1) [1]. Thus, reducing the area and power overhead by processing units in PIM architectures is critical for mitigating the computation–capacity trade-off.

One of the promising approaches to addressing the aforementioned challenge is to simplify the arithmetic units embedded inside memory. Among various arithmetic representations, the logarithmic number system (LNS) has been recognized as an attractive alternative to conventional floating-point arithmetic since it can fundamentally reduce arithmetic complexity [2], [3]. Unlike conventional floating-point arithmetic, LNS represents numbers in the logarithmic domain, turning multiplication and division into simple addition and subtraction. Consequently, complex floating-point multipliers can be replaced with simpler arithmetic hardware, offering substantial reductions in area and power consumption.

Taewoon Kang, Yujin Kim, and Gunjae Koo are with the Department of Computer Science and Engineering, Korea University, Seoul, 02841, South Korea (e-mail: taewoon_kang@korea.ac.kr; yujin_kim@korea.ac.kr; gunjaekoo@korea.ac.kr). Jonghyun Lee, and Murali Annavaram are with the Ming-Hsieh Department of Electrical Engineering, University of Southern California, Los Angeles, CA 90089, USA (e-mail: leejongh@usc.edu; annavara@usc.edu). Gunjae Koo is the corresponding author.

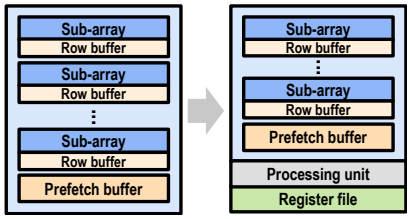


Fig. 1. HBM-based PIM sub-array loss

In LNS, a real-valued number is represented as:

$$X \to \begin{cases} x = \log_b |X|, \\ s = \begin{cases} 0, & \text{if } X > 0, \\ 1, & \text{if } X < 0, \end{cases} \end{cases}$$

where x denotes the logarithmic magnitude and s denotes the sign bit. Since zero has no finite logarithmic representation, LogPIM assigns it a reserved codeword in the logarithmic field. For nonzero operands, arithmetic operations in the logarithmic domain are expressed as follows:

$$\log_b(x \pm y) = \log_b x + \log_b\left(1 \pm b^{\log_b y - \log_b x}\right),$$
$$\log_b(x \times y) = \log_b x + \log_b y,$$
$$\log_b(x \div y) = \log_b x - \log_b y.$$

The primary limitation of LNS is that addition and subtraction require evaluating nonlinear functions, which are typically implemented using lookup tables (LUTs). Although LUT access cost and area overhead have limited the adoption of LNS in conventional processors, LNS arithmetic is particularly well suited to PIM architectures. Since PIM inherently integrates memory arrays with computation units, LUTs can be stored inside DRAM cells with negligible storage overhead. Thus, PIM provides a promising platform for exploiting the computational advantages of LNS while efficiently supporting LUT-assisted arithmetic.

We propose **LogPIM**, a LNS-based PIM architecture that simplifies conventional PIM processing units. LogPIM replaces complex FP16 arithmetic units with dual-mode SIMD adders and lightweight LUT controllers. This design substantially reduces area and power overhead, mitigating the memory capacity loss of conventional HBM-PIM architectures while maintaining high-performance in-memory computation.

II. System overview

A. Proposed architecture

Fig. 2 presents the overall architecture of LogPIM. To reduce the hardware overhead of conventional HBM-PIM processing units, LogPIM eliminates the dedicated SIMD FP16 multiplier, redesigns the conventional SIMD FP16 adder as

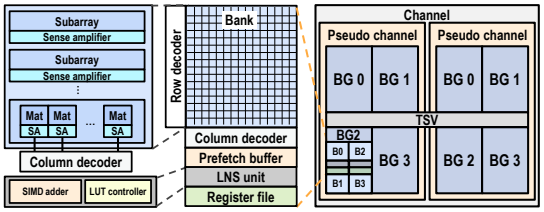
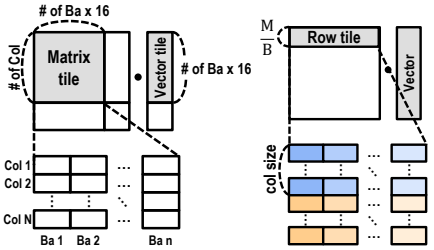


Fig. 2. Overall architecture



(a) Previous work (b) LogPIM

Fig. 3. Comparison of tiling methods for outer-product GeMV

a dual-mode SIMD adder, and introduces a lightweight LUT controller. The proposed SIMD adder supports both FP16 addition and 16-bit fixed-point addition. Since it processes only one numerical format at a time, both modes share as much of the arithmetic datapath as possible, including the significand or fixed-point addition logic, normalization-related components, and register interfaces. The dual-mode SIMD adder includes a small combinational logic that selects between FP16 and fixed-point operation, without introducing an additional cycle compared to the baseline SIMD multiplication. This hardware reuse reduces the additional area and power required for supporting both logarithmic-domain multiplication and FP16 accumulation. For the register files, LogPIM retains the same configuration as the baseline HBM-PIM architecture [1], requiring no additional register storage or modification to the existing register organization.

The LUT controller converts LNS products into FP16 values. For each generated LNS product, the controller extracts the LUT index, derives the corresponding DRAM column address within the LUT row, and tracks the addresses of pending requests. To avoid introducing a dedicated tracking buffer, the controller stores these addresses in the unused portion of the existing SRF (scalar register file), requiring no additional register-file capacity. Through tracking, the LUT controller groups requests targeting the same DRAM column and serves them through fine-grained column accesses to the LUT row. Because multiple conversion requests can share the same row activation and, when their LUT entries reside in the same DRAM column, the controller amortizes DRAM access overhead while avoiding a dedicated on-chip LUT structure.

LogPIM adopts a 16-bit base-2 LNS format comprising a 1-bit sign field for X and a 15-bit signed fixed-point field for $\ell = \log_2 |X|$. The logarithmic field uses a two's-complement I6F9 representation, in which the six integer bits include the sign bit of ℓ, and the remaining nine bits represent its fractional component. The 9-bit fractional field indexes 512 16-bit entries storing 2^F, yielding a 1-KB conversion LUT that fits in one DRAM row. During input encoding, zero and magnitudes below the minimum nonzero magnitude representable by LNS16 are mapped to the reserved zero codeword. During LNS multiplication, a zero operand directly produces a zero product; otherwise, an underflowed logarithmic sum is mapped to zero, whereas an overflowed sum is saturated to the maximum representable I6F9 value.

For a nonzero product within the representable I6F9 range, the logarithmic magnitude is decomposed as $\ell = I + F$, where I denotes the signed integer component and $F \in [0, 1)$ denotes the fractional component. The LUT is indexed by F to obtain the fractional anti-logarithm 2^F, while I scales the LUT output through a left or right shift. The reconstructed magnitude is then bounded to the supported FP16 range, and the value-sign bit is applied to generate the signed result. This decomposition supports a wide LNS dynamic range while limiting the LUT storage requirement to one DRAM row per bank.

At the system boundary, we assume a heterogeneous host–PIM execution flow in which a host accelerator, such as a GPU or NPU, converts each FP16 input feature vector to LNS16 before dispatching it to LogPIM. Model weights are converted to LNS16 offline and stored in HBM in the encoded format. Within LogPIM, each LNS product is converted to FP16 before accumulation; therefore, the completed GeMV output is already in FP16 and can be fetched directly by the host accelerator for subsequent operations, such as softmax and reduction.

B. Tiling

Fig. 3 compares the GeMV data-placement schemes. While previous outer-product PIMs like AESPA [4] partition matrices along the reduction dimension (Fig. 3a) and require a bank-group-level shared accumulator to merge partial sums, LogPIM fundamentally eliminates this overhead through an output-row partitioned tiling scheme (Fig. 3b).

Given that matrix row sizes are typically multiples of the bank count (e.g., $B = 128$), we assign each bank a disjoint set of output rows ($\mathcal{R}_b$) and store the corresponding matrix data in a column-wise layout mapped to DRAM columns. Upon receiving a broadcast vector element, each bank performs element-wise multiplication with its local column data and accumulates the products directly into its local registers. Because every term required for a specific output row resides entirely within its assigned bank, each bank independently produces a complete portion of the final result vector. This strictly local accumulation completely removes the necessity for cross-bank reduction and shared accumulators.

C. Basic Execution Flow

Fig. 4 illustrates the basic per-bank execution flow.Each bank reserves one DRAM row for the LUT used for LNS-to-FP16 conversion. The LUT row is stored alongside the tiled matrix rows and accessed through the existing row buffer and sense amplifiers, requiring no dedicated LUT storage structure and incurring only one-row capacity overhead per bank.

At outer-product iteration j, a tiled matrix row is first activated, and the broadcast vector element x_j is multiplied by the corresponding matrix elements. Since both operands are represented in LNS, each multiplication is reduced to a fixed-point addition performed by the SIMD adder. The register file temporarily stores the resulting LNS products, while the LUT

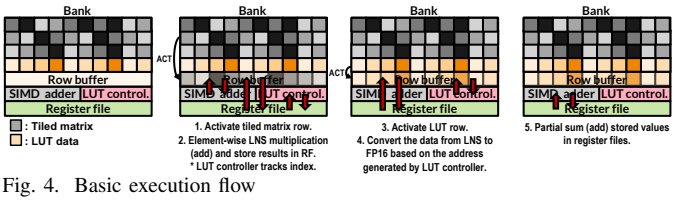


Fig. 4. Basic execution flow

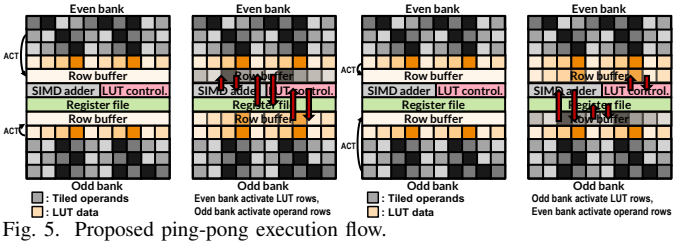


Fig. 5. Proposed ping-pong execution flow.

controller generates and tracks the corresponding LUT indices. The bank then activates the reserved LUT row and converts the buffered LNS products to FP16. Multiple conversion requests can be grouped so that one LUT-row activation is shared across several LUT column accesses. The converted products are subsequently accumulated in FP16 using the same SIMD adder datapath.

However, this execution flow incurs frequent row conflicts between matrix accesses and LUT accesses. Since the tiled matrix and LUT entries reside in different DRAM rows of the same bank, each conversion phase requires the currently open matrix row to be precharged before activating the LUT row. The bank must subsequently reactivate a matrix row to continue operand processing. Therefore, although batching amortizes a LUT-row activation across multiple lookup requests, repeated transitions between operand and LUT rows remain on the critical execution path. To eliminate these repeated matrix-to-LUT row transitions, LogPIM introduces a ping-pong execution scheme that pipelines operand and LUT accesses across the two banks of each even–odd bank pair.

D. Ping-Pong Execution Mechanism

Fig. 5 illustrates the proposed ping-pong execution flow across an even–odd bank pair sharing a single PU. At any given phase, one bank operates as the *operand bank*, while the other bank operates as the *LUT bank*. Without loss of generality, we assume that the even bank initially acts as the LUT bank and the odd bank as the operand bank. For each outer-product iteration j, the host broadcasts the vector element x_j to all PUs, and a 32-byte column access retrieves 16 LNS-encoded matrix elements from the active operand row. The SIMD adder performs the element-wise LNS multiplications as fixed-point additions, and the resulting products are buffered in the register file. Meanwhile, the LUT controller generates the corresponding LUT addresses from the buffered products.

Unlike the basic execution flow, the operand bank does not close its matrix row for number-system conversion. Instead, the generated lookup requests are served by the already-open LUT row in the paired bank. Because the LUT row remains active across multiple operand accesses, the controller can issue consecutive LUT column accesses without additional row activations. Even when buffered lookup requests are distributed across different columns, these accesses overlap with SIMD addition in the paired operand bank, hiding most of the lookup latency. Operand accesses and LUT accesses can therefore be pipelined across the bank pair, hiding a significant portion of the conversion latency while avoiding repeated matrix-to-LUT row conflicts.

After conversion, the FP16 products are accumulated using the same SIMD adder datapath, which is time-multiplexed between fixed-point mode for LNS multiplication and FP16 mode for accumulation.

Once the operand-bank tiles scheduled for the current phase are processed, the two banks exchange their roles. The odd bank activates its LUT row, while the even bank activates a tiled matrix row, and the same execution flow is repeated for the even-bank operands. Thus, neither bank is permanently dedicated to LUT service, and both banks continue to store matrix tiles and their corresponding output elements. Although role exchange requires precharge and activation operations, unlike the basic execution scheme, which frequently switches rows between operand-processing and lookup phases, these operations occur only after processing an entire matrix tile. Moreover, the two banks can issue transition commands in parallel. Since each even–odd bank pair already shares one PU, using the non-operand bank for LUT service does not reduce the number of available PUs.

At the final outer-product iteration, the accumulated result is returned to the host in FP16. Returning the FP16 result directly allows subsequent linear operations or LLM layers to consume the output without additional format conversion.

III. Evaluation

We evaluate LogPIM using an in-house cycle-accurate simulator based on DRAMsim3 [5]. We assume that the input vector and matrix weights are pre-encoded in LNS before GeMV execution. Thus, the reported latency and energy cover the PIM kernel only and exclude host-side FP16-to-LNS16 encoding of the input vector. The accumulated output is returned in FP16 so that it can be directly consumed by another processing unit for subsequent linear operations or by the host for operations not supported in memory. We implement the baseline and proposed logic in RTL and synthesize them using Synopsys Design Compiler with the SAED 14-nm FinFET technology at 250 MHz, the same operating frequency as HBM-PIM. The reported PU area includes the arithmetic units, LUT controller, GRF, SRF, and CRF. For energy estimation, we use the DRAMSim3 HBM2 model, including ACT, PRE, RD, and background energy. We calculate logic energy from post-synthesis power and simulated activity, including the LUT controller and register files.

A. Performance

Fig. 6 presents the performance comparison. We compare LogPIM with HBM-PIM [1], an idealized HBM-PIM configuration that activates all banks with a single command, the PIM-only execution scheme of ComPASS [6], and AESPA [4]. For a fair comparison, we evaluate all architectures using the same JEDEC HBM2 timing parameters [7], an eight-die

HBM2 stack with four PIM-enabled dies, and the register file configuration from HBM-PIM [1]. Across all evaluated matrix sizes, ping-pong LogPIM achieves average speedups of $1.68\times$, $1.54\times$, and $1.30\times$ over HBM-PIM, ComPASS, and AESPA, respectively. It also achieves average speedups of $1.24\times$ over the idealized all-bank HBM-PIM configuration and $3.36\times$ over the basic LogPIM execution flow. The speedups remain nearly constant across matrix sizes because each architecture executes a fixed command sequence per matrix tile, while the number of tile iterations scales proportionally with matrix size. Applying the same data placement and scheduling to conventional FP16 MAC achieves a $1.69\times$ average speedup over HBM-PIM, nearly identical to the $1.68\times$ of ping-pong LogPIM. Thus, the speedup primarily comes from the execution organization, while LNS mainly reduces PU hardware overhead.

The basic execution flow of LogPIM requires approximately $2.00\times$ the execution time of HBM-PIM, primarily because the matrix and LUT reside in the same bank and therefore require repeated row switching. Each lookup precharges the matrix row, activates the LUT row, performs the lookup, and then reactivates the matrix row. Although batching eight LNS products amortizes the LUT activation overhead, it cannot eliminate these row transitions. Separate execution of the even and odd bank sets and command dependencies further prevent matrix accesses from overlapping with LNS-to-FP16 conversion. In contrast, HBM-PIM keeps the matrix row active during MAC operations and accumulates partial sums in local registers.

B. Area and Energy

The area and energy results are normalized to HBM-PIM. Including the GRF, SRF, and CRF, LogPIM reduces the processing-unit area by 30.1%. This reduction comes primarily from eliminating the dedicated FP16 multiplier and replacing the conventional FP16 adder with a dual-mode SIMD adder that supports both FP16 and fixed-point addition through shared datapath resources. Although the GRF, SRF, and CRF remain unchanged from HBM-PIM and constitute a substantial fraction of the processing-unit area, LogPIM still achieves a considerable area reduction. As a first-order proportional estimate, this reduces the 50% capacity loss of a PIM-enabled die to approximately 35%, increasing the evaluated eight-die stack capacity from 6.0 GB to approximately 6.6 GB; actual recovery depends on physical layout granularity. To support LNS-to-FP16 lookup operations, LogPIM reserves one DRAM row per bank for the conversion LUT, incurring a storage overhead of only one row per bank.

The basic LogPIM execution flow consumes $3.56\times$ the modeled GeMV energy of HBM-PIM. Repeated transitions between the matrix and LUT rows incur additional precharge and activation energy and extend the execution time, thereby increasing background energy. Ping-pong LogPIM avoids these repeated transitions within the inner execution flow by overlapping matrix accesses and LUT operations across paired banks. Consequently, ping-pong LogPIM reduces the modeled GeMV energy by 33.9% relative to HBM-PIM.

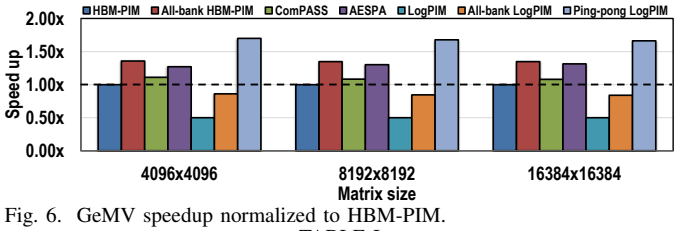


Fig. 6. GeMV speedup normalized to HBM-PIM.

TABLE I
AVERAGE GEMV OUTPUT ERROR COMPARED WITH THE FP32 REFERENCE ACROSS ALL EVALUATED MATRIX SIZES.

Format	MSE	MAE	MRE	Cosine
FP16	7.32×10^{-4}	2.04×10^{-2}	4.18×10^{-3}	≈ 1.0000
LNS16 (I6F9)	1.05×10^{-2}	7.73×10^{-2}	1.30×10^{-2}	≈ 0.9999
INT8	1.88	1.03	9.49×10^{-2}	≈ 0.9998
FP8 (E4M3)	1.19×10^{1}	2.60	2.53×10^{-1}	≈ 0.9993

C. Numerical Accuracy

Table I compares the GeMV output errors of INT8, FP8, FP16, and LNS16 against the FP32 reference. For each format, the reported error metrics are averaged across the same matrix sizes used in the performance evaluation. LNS16 uses a 16-bit format consisting of a 1-bit sign, a 6-bit integer, and a 9-bit fraction. LNS16 exhibits substantially lower errors than INT8 and FP8 and maintains a cosine similarity of approximately 0.9999 to FP32, although its absolute errors remain higher than FP16. Thus, the current evaluation demonstrates GeMV-level numerical fidelity rather than end-to-end model accuracy.

IV. CONCLUSION

We present LogPIM, an LNS-based PIM architecture that reduces the hardware overhead of HBM-based PIM by eliminating the FP16 multipliers and supporting both LNS multiplication and FP16 accumulation with dual-mode SIMD adders. A DRAM-resident LUT and the proposed ping-pong scheme enable efficient LNS conversion while avoiding repeated matrix–LUT row switching. Our evaluation demonstrates that LogPIM achieves low hardware and energy overhead while improving performance and maintaining high GeMV-level numerical fidelity. As future work, we plan to extend LogPIM to a heterogeneous NPU–PIM system with native LNS support and dynamic, format-adaptive LNS execution.

REFERENCES

[1] S. Lee *et al.*, "Hardware architecture and software stack for pim based on commercial dram technology : Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 43–56.

[2] Y. Kim *et al.*, "Logflex: Flexible-bit log arithmetic accelerator for language models on edge," *IEEE Micro*, 2026.

[3] P. Haghi *et al.*, "Bridging the gap between llms and lns with dynamic data format and architecture codesign," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 1617–1631.

[4] H. Kal *et al.*, "Aespa: Asynchronous execution scheme to exploit bank-level parallelism of processing-in-memory," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. ACM, 2023, p. 815–827.

[5] S. Li *et al.*, "Dramsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.

[6] S. Yu *et al.*, "Compass: A compatible pim protocol architecture and scheduling solution for processor-pim collaboration," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 49–62.

[7] JEDEC Solid State Technology Association, "High bandwidth memory (hbm) dram (hbm1, hbm2)," JEDEC, Tech. Rep. JESD235D, 2020, jEDEC Standard.